Pedidos e Catálogo

Evolução arquitetural incremental, sem downtime e sem quebrar quem já usa

90 dias

app, marketplace e 2° país

10x

tráfego projetado

99,9%

disponibilidade mensal

500 ms

p95 na criação de pedido

Desafio técnico · COE Arquitetura de Soluções

O que muda em 90 dias e o que não pode quebrar

Novos consumidores

1. **App móvel**
   Novo canal, precisa de API estável e retry seguro em rede ruim

2. **Parceiros de marketplace**
   API pública versionada e notificação assíncrona de status

3. **Segundo país**
   Moeda, preço por país e dados pessoais no próprio país

Restrições

6 meses
contratos atuais continuam compatíveis

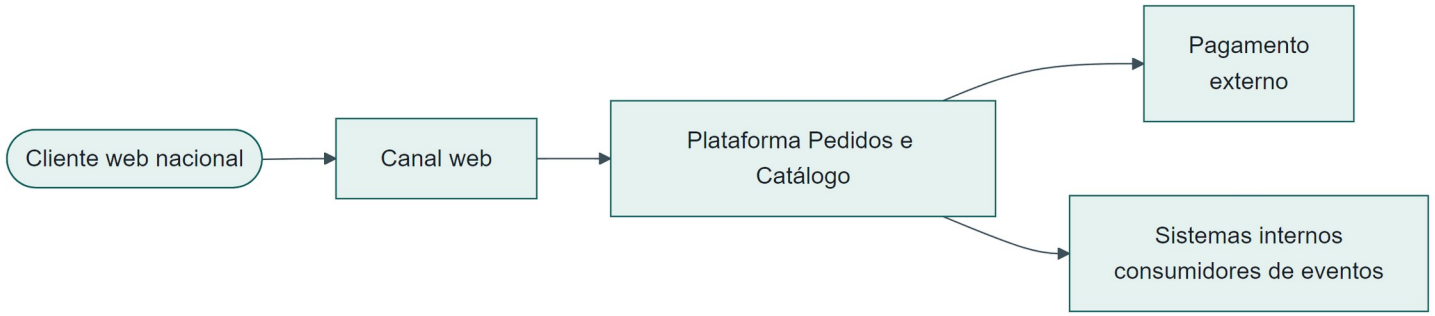
30 dias
primeira melhoria segura em produção, sem janela

10x · 99,9% · 500 ms
escala, disponibilidade e latência

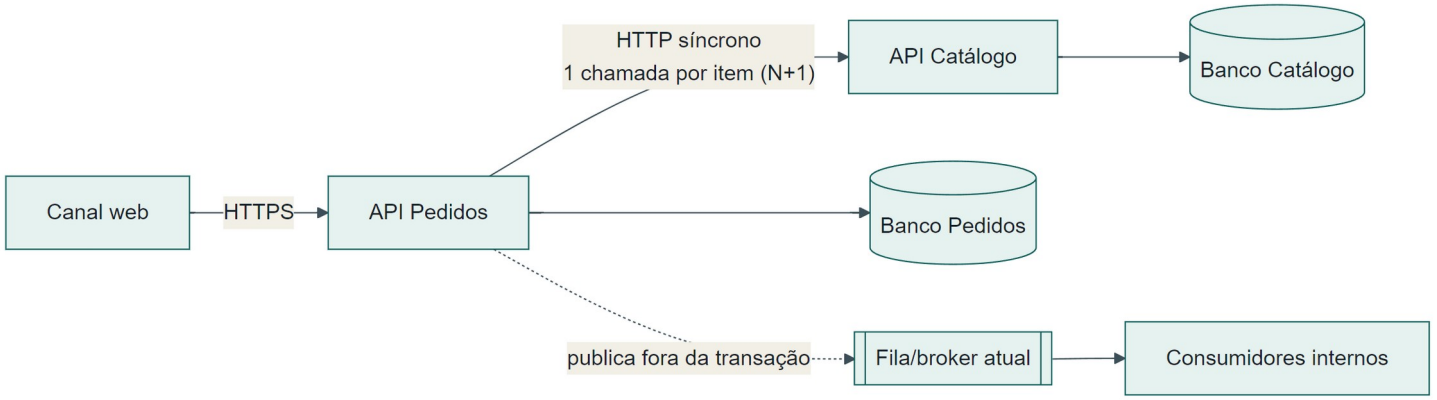
LGPD
e residência de dados do novo país

Arquitetura atual (C4: contexto e contêineres)

Contexto



Contêineres



Pontos de falha

Catálogo lento

derruba a criação (cascata)

Timeout + retry

pedido duplicado

Commit e publish separados

evento perdido ou fantasma

Banco único

para tudo

Onde dói quando o tráfego multiplica por 10

P1

Alto

Retry sem idempotency key

Pedido e cobrança em dobro. Mais tráfego, mais timeout, mais retry

P2

Alto

Evento fora da transação

Parceiro recebe evento de pedido que não existe, ou não recebe nada

P3

Alto

N+1 síncrono no Catálogo

p95 cresce com o nº de itens; Catálogo fora derruba Pedidos

P4

Médio

Sem snapshot de preço

Pedido "muda" quando o Catálogo muda. Sem auditoria

P5

Alto

Sem versionamento de API

Qualquer mudança para app ou parceiro pode quebrar o web

P6

Alto

Dados pessoais sem separação por país

Bloqueia a entrada no 2º país

Decisão crítica: criar pedido sem duplicar e sem quebrar ninguém

O que a fatia prova

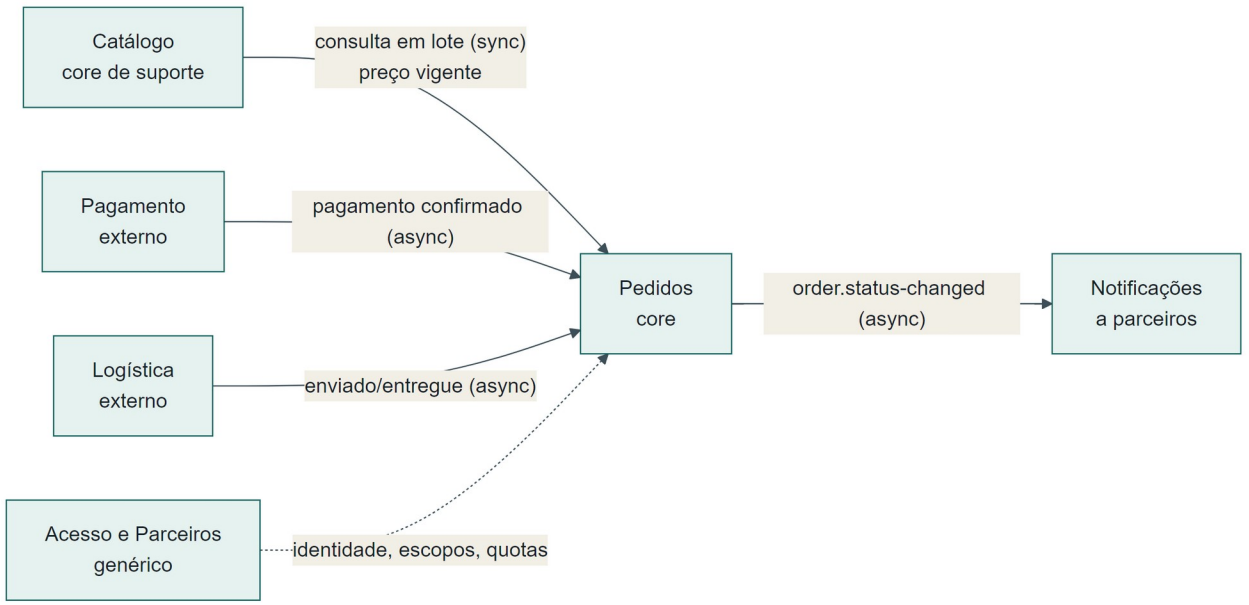
- Idempotency-Key: mesma chave não cria outro pedido
- /v1 igual ao de hoje, /v2 novo, lado a lado
- Snapshot de preço e descrição no item
- Pedido, chave e outbox na mesma transação

Ataca P1, P2, P4 e P5 dentro do serviço de Pedidos, sem infraestrutura nova. Cabe em 30 dias e é pré-requisito do resto: sem idempotência, qualquer retry que a gente colocar depois aumenta a duplicidade.

Por que não começar por outra coisa

Alternativa	Por que não agora
Separar Catálogo de Pedidos	Projeto grande, não entrega em 30 dias e não resolve duplicidade
Strangler Fig com gateway	Entra no plano (é como v1 e v2 convivem), mas sozinho não prova consistência
Cache / lote no Catálogo	Resolve latência (P3). É a entrega dos 60 dias
Outbox com relay e broker	A gravação entra agora; relay e broker dependem da escolha de infraestrutura

Bounded contexts: quem é dono de quê

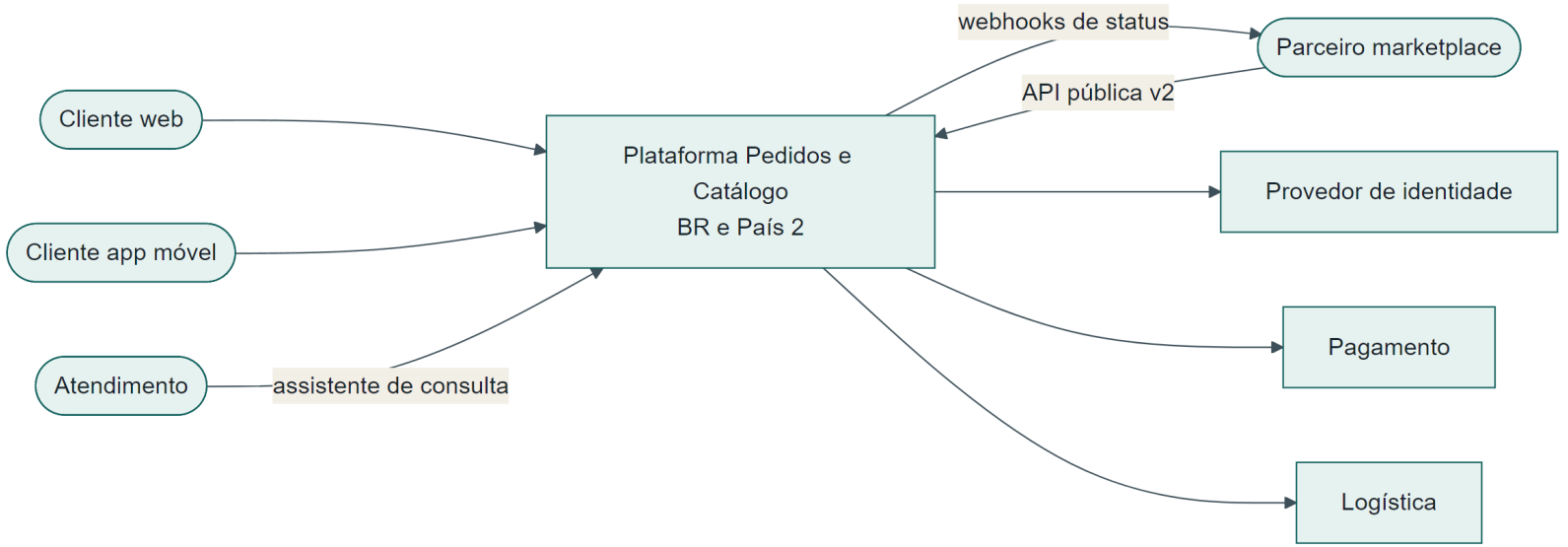


Contexto	Possui	Não possui
Catálogo	Produto, preço vigente por país	Pedido, cliente
Pedidos (core)	Pedido, item + snapshot, status, chave, outbox	Preço vigente, pagamento
Notificações	Assinaturas, entregas, DLQ	Dados do pedido além de ID/status
Acesso e Parceiros	Clientes de API, escopos, quotas, país	–

- Cada contexto é dono do seu banco. Ninguém lê o banco do outro
- Pedidos copia o que precisa do Catálogo (snapshot), não guarda referência viva de preço
- Cadastro do cliente fica em Clientes/Conta; Pedidos guarda só contato e entrega do pedido, no país do pedido
- Eventos e webhooks levam só IDs e status

Catálogo é upstream de Pedidos. Pagamento e Logística entram por camada anticorrupção. Parceiros consomem a linguagem publicada (OpenAPI v2 + AsyncAPI).

Arquitetura-alvo: contexto

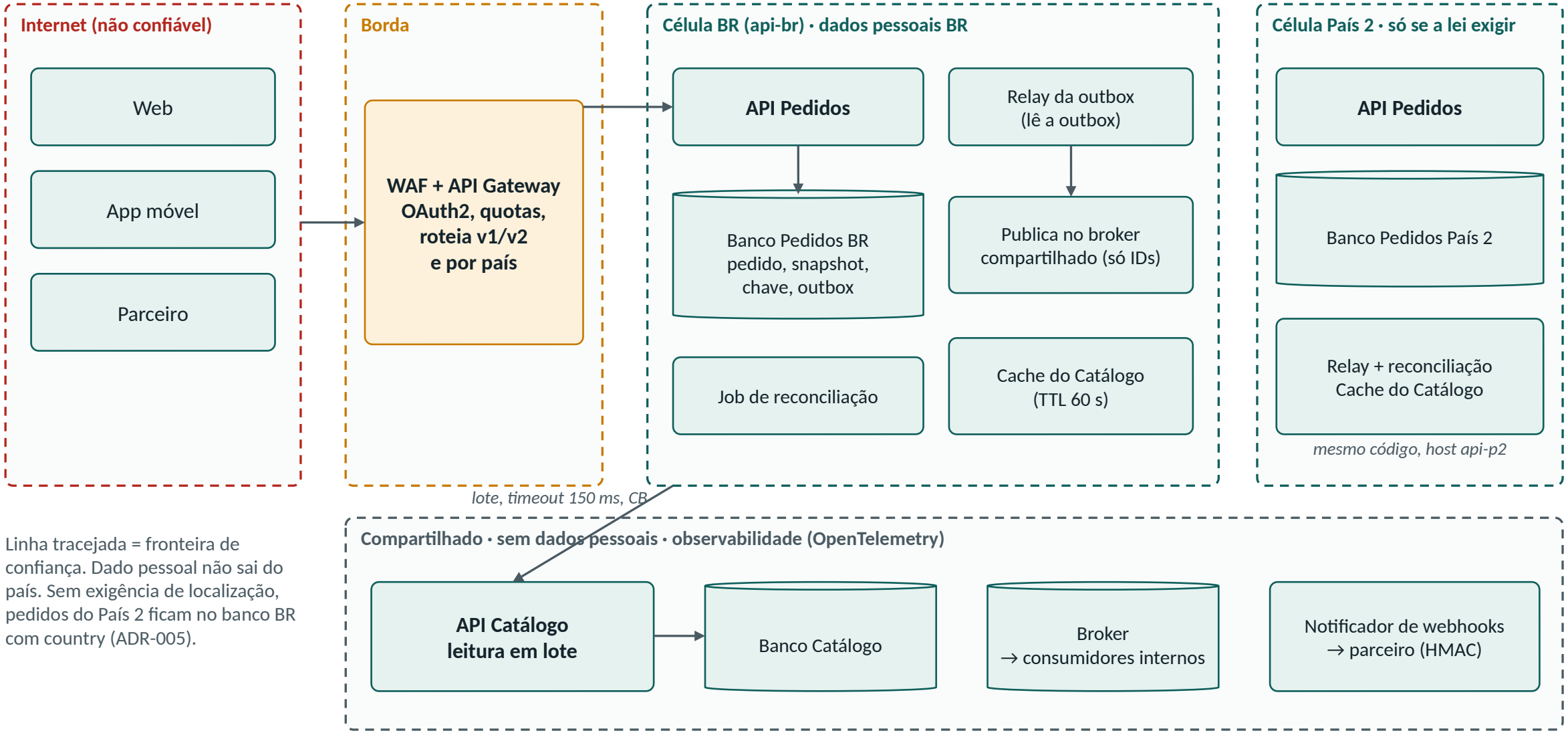


O que entra

- App móvel e parceiros pela API v2
- Webhooks de status para parceiros
- Provedor de identidade (OAuth2/OIDC)
- Assistente de consulta para o atendimento
- Mesma plataforma em BR e País 2

Mesmo sistema, evoluído. Sem reescrita.

Arquitetura-alvo: contêineres



Linha tracejada = fronteira de confiança. Dado pessoal não sai do país. Sem exigência de localização, pedidos do País 2 ficam no banco BR com country (ADR-005).

Fronteiras de confiança e pontos de falha

Fronteiras de confiança

Fronteira	Controle
Internet → Borda	TLS 1.2+, WAF, OAuth2/JWT, quotas
Borda → Célula	mTLS, claims (cliente, escopo, país)
Célula → Compartilhado	Só IDs; dado pessoal não sai da célula
Célula → Parceiro	Webhook HMAC, só ID e status, URL validada
Célula → Observabilidade	Mascaramento de dado pessoal no coletor

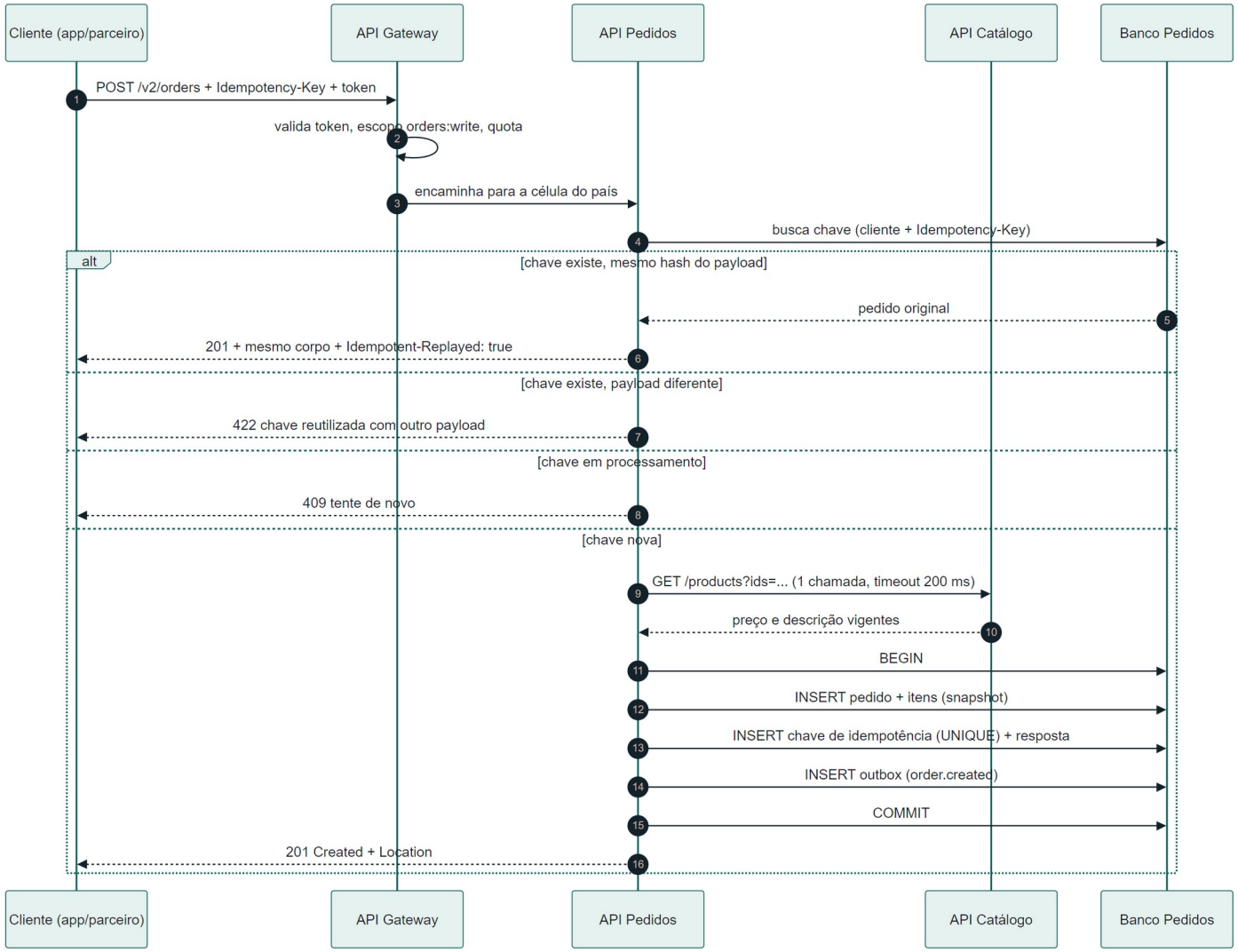
Pontos de falha no alvo

Falha	O que acontece
Catálogo lento ou fora	Usa cache; sem cache, 503 + Retry-After (seguro pela idempotência)
Broker fora	Outbox acumula; pedidos continuam sendo criados
Parceiro fora	Retry e DLQ; parceiro reconcilia pela API
Banco da célula	Failover para réplica; a outra célula segue
Gateway	Múltiplas zonas

Cada atributo de qualidade tem um mecanismo

Atributo	Mecanismo	Como medimos
Escala 10x	Catálogo em lote + cache, Pedidos stateless com autoescala	200 criações/s em teste de carga
Disponibilidade 99,9%	Multi-zona, outbox desacopla o broker, circuit breaker, canário + feature flag	≤ 43 min de 5xx por mês
p95 500 ms	1 chamada em lote ao Catálogo, cache, orçamento de latência	p95 ≤ 500 ms no gateway
Consistência	Idempotency-Key, snapshot + outbox na mesma transação, dedup por eventId	0 duplicatas, 0 eventos perdidos
Auditabilidade	Snapshot de preço, histórico de status com autor	100% dos itens com snapshot
Segurança / LGPD	OAuth2 com escopos, coluna de país (célula só se a lei exigir), eventos sem dado pessoal	404 para recurso alheio, 429 na quota
Evolução sem quebra	/v1 e /v2 em paralelo, teste de contrato no CI	Teste do consumidor v1 verde

Sequência: criação de pedido (v2)



Quatro respostas possíveis

201 + mesmo corpo

chave existe, mesmo payload (replay)

422

chave existe, payload diferente

409

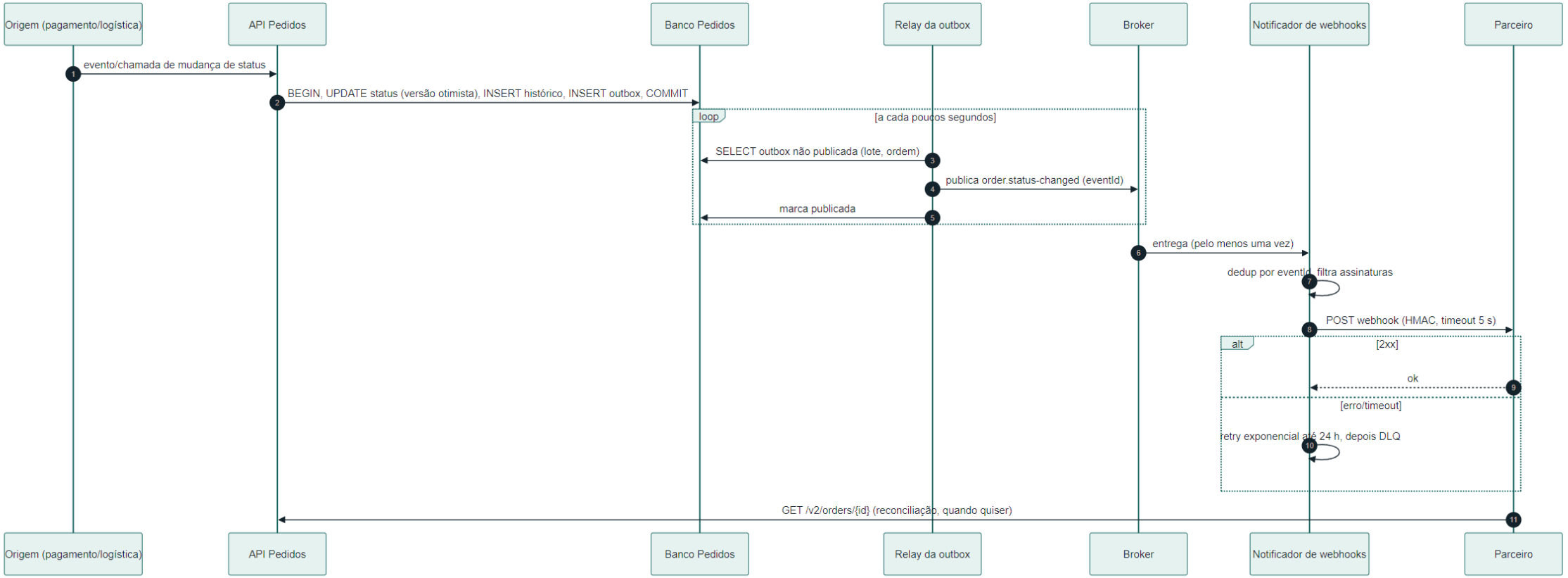
chave ainda em processamento

201 Created

chave nova: pedido, chave e outbox no mesmo COMMIT

Duas chamadas ao mesmo tempo: a UNIQUE derruba uma; ela relê e devolve o pedido original.

Sequência: mudança de status e notificação



Pelo menos uma vez

Consumidor deduplica por eventId

Ordem por pedido

Partição por orderId; campo version descarta evento velho

Parceiro fora

Retry exponencial até 24 h, depois DLQ

Reconciliação

Parceiro consulta GET /v2/orders/{id} ou GET /v2/orders?updatedSince=

Síncrono ou assíncrono: regra simples

Síncrono só quando a resposta depende do dado e o dado precisa estar atual. O resto é assíncrono.

Interação	Estilo	Por quê
Cliente → criar pedido	Síncrono	Cliente precisa do ID e do total. A chave de idempotência torna o retry seguro
Cliente → consultar pedido	Síncrono	Leitura simples, dado local da célula
Pedidos → Catálogo (preço)	Síncrono, em lote	Preço precisa estar certo na hora. Lote + cache resolve N+1 e latência
Pedidos → parceiros e sistemas internos	Assíncrono	Outbox → broker → webhook. Parceiro lento não afeta a criação
Pagamento / Logística → Pedidos	Assíncrono	Origem externa com retry próprio; dedup pelo ID da origem

Decisões (ADRs): consistência e contrato

ADR-001

Idempotência na criação

Decisão: Idempotency-Key do cliente, gravada no banco com hash do payload

Descartado: Cache Redis (perde garantia e fica fora da transação); dedup por conteúdo (falso positivo)

ADR-002

Outbox transacional

Decisão: Evento gravado na mesma transação; relay publica depois

Descartado: Publicar após commit (perde evento); 2PC (acopla e é lento); CDC fica como evolução

ADR-003

Snapshot + Catálogo em lote

Decisão: Uma chamada por pedido, cache de 60 s, preço gravado no item

Descartado: Réplica local do Catálogo (preço velho, mais um modelo para manter)

ADR-004

Versão na URL, v1 e v2 juntos

Decisão: /v1 intacto, /v2 novo, mesmo backend. v1 vive ≥ 6 meses

Descartado: Mudar no lugar (quebra o web); versão por header (pior para parceiro e gateway)

Decisões (ADRs): país, parceiros e acesso

ADR-005

Coluna de país ou célula mínima

Decisão: país vem do client_id; célula mínima (API + banco na região) só se a lei exigir. Broker e Catálogo compartilhados

Descartado: célula completa com broker e notificador por país (custo sem ganho de conformidade); plataforma inteira duplicada

ADR-006

Webhooks assinados

Decisão: HTTPS + HMAC, retry 24 h, DLQ, fila por parceiro

Descartado: Parceiro direto no broker (expõe infraestrutura); só polling (carga e atraso)

ADR-007

OAuth2 + quotas no gateway

Decisão: client_credentials, JWT de 15 min, escopos, quota por cliente

Descartado: API key estática (sem expiração nem escopo); mTLS para todos (operação pesada)

ADR-005 está como Proposto: depende de parecer jurídico do País 2.

Resiliência: como evitar o efeito cascata

1. **Timeout**

   150 ms no Catálogo (sem retry interno), 2 s no gateway, 5 s no webhook

2. **Retry com backoff**

   Só em operação idempotente, com teto e jitter. Uma camada só repete

3. **Circuit breaker**

   Abre com 50% de falha em 20 chamadas; meio-aberto após 30 s

4. **Bulkhead**

   Pool separado para criar e consultar; fila e worker por parceiro

5. **Degradação controlada**

   Catálogo fora → último preço em cache por até 15 min; sem cache → 503 + Retry-After. Broker fora → outbox acumula

Orçamento de latência (p95)

Etapa	ms
Gateway (TLS, token, quota)	30
Leitura da chave	10
Catálogo em lote (sem cache, sem retry)	150
Transação	60
Serialização e rede	50
Total	300

Pior caso 300 ms: sobram 200 ms para a meta de 500 ms. Com cache quente, o Catálogo cai para menos de 5 ms.

Segurança e LGPD: threat model (STRIDE)

Área	Ameaça principal	Controle	Quando
Identidade	Vazamento de client_secret do parceiro	Token de 15 min, rotação, escopo mínimo	60 dias
API pública	Parceiro lê pedido de outro (BOLA)	Autorização por recurso, 404 para pedido alheio	Provado na fatia
API pública	Replay para duplicar pedido	Idempotency-Key por cliente	30 dias
API pública	Preço manipulado no payload	Preço nunca vem do cliente: vem do Catálogo	30 dias
Dados	Dado pessoal fora do país	Coluna de país (célula se a lei exigir); eventos e logs só com IDs	90 dias
Eventos	Webhook falso em nosso nome	HMAC com timestamp, janela de 5 min	60 dias
Dependências	SSRF pela URL de webhook	Só HTTPS, bloqueio de IP privado, egress controlado	60 dias

25 ameaças mapeadas em docs/05-threat-model.md (inclui IA). LGPD: exclusão por pseudonimização, mantendo o registro fiscal.

Plano 30 / 60 / 90 dias

30 Parar de errar

- Idempotency-Key opcional no /v1
- Snapshot de preço
- Outbox na transação (dual write)
- OpenTelemetry e SLOs
- /v2 implantado, fechado

60 Escalar e abrir

- Catálogo em lote + cache + CB
- Relay da outbox + broker
- Gateway OAuth2, quotas, /v1 e /v2
- App em produção no /v2
- Webhooks com parceiros piloto

90 Expandir

- País 2: coluna de país ou célula mínima (parecer)
- Parceiros em produção com quotas
- Depreciação do /v1 anunciada (≥ 6 meses)

Rollback: toda entrega atrás de feature flag. Migrações só aditivas. Dupla publicação até o último consumidor migrar.

A prova: um comando, 52 testes

```
dotnet test
```

Dados sintéticos em data/catalogo-sintetico.json. SQLite, sem Docker.

Pipeline (GitHub Actions)

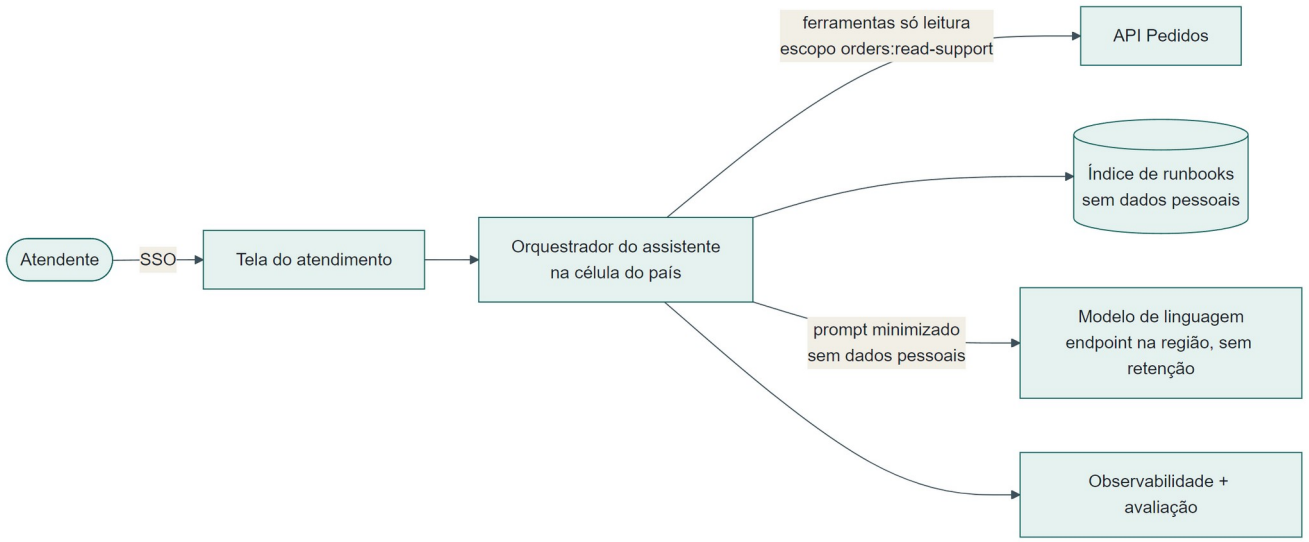
- Build e testes
- Spectral: lint da OpenAPI
- AsyncAPI CLI: valida o schema de status
- oasdiff: barra breaking change no contrato
- NetArchTest: regras de camadas
- Checagem de ADRs e exceções vencidas

O teste prova que...	Tipo
Mesma chave + mesmo payload devolve o pedido original	positivo
Mesma chave + payload diferente → 422, nada criado	negativo
v2 sem chave → 400, nada criado	negativo
Chamadas concorrentes com a mesma chave → 1 pedido	positivo
Consumidor web atual cria e consulta no v1 como antes	contrato
v1 sem chave mantém o comportamento antigo	contrato
Evento vai para a outbox junto com o pedido	positivo
Replay e requisição rejeitada não geram evento	negativo
Eventos não carregam dado pessoal	fitness

Trade-offs aceitos

Escolha	O que ganhamos	Custo aceito
Chave no banco, não em cache	Garantia na mesma transação do pedido	Uma escrita a mais por criação
Outbox, não publicação direta	Evento existe se e só se o pedido existe	Atraso de segundos no evento
Versão na URL	Roteamento, métrica e desligamento simples	Duas versões ativas por ≥ 6 meses
Coluna de país; célula mínima só se a lei exigir	Residência sem gastar antes do parecer jurídico	Escolha final depende do jurídico até o dia 30
Preço do momento do pedido + cache	Latência estável e auditoria	Preço até 60 s defasado
Evoluir, não reescrever	Entrega em 30 dias, risco baixo	Convive com parte do legado por mais tempo

IA: assistente de consulta para o atendimento



Controle	Como
Isolamento	Um orquestrador por célula, modelo na mesma região, sem retenção
Minimização	Nome, documento e endereço nunca vão para o prompt
Guardrails	Só ferramentas de leitura; dado de pedido nunca vira instrução; sem fonte, responde "não sei"
Avaliação	200 perguntas sintéticas; exatidão ≥ 90%, 0 vazamento

Uso de IA neste desafio

Prompts, validações e decisões rejeitadas estão em docs/uso-de-ia.md. Toda saída foi conferida contra o requisito e pelos testes.

Investimento, riscos e o que depende do cliente

~116

pessoa-dia na fase de 30 dias

~R$ 93 mil

custo da fase 1 sem IA (~R$ 79 mil com IA)

~5,25 FTE

arquiteto, TL, 2 devs, QA, SRE

Riscos

- Tráfego real desconhecido (premissa 20 → 200 criações/s)
- Regra de residência do 2° país indefinida
- Consumidores de eventos não mapeados
- Web demorar a mandar a chave

Validar com o cliente

- Baseline de tráfego e contrato v1 real
- País e parecer jurídico (LGPD + lei local)
- Preço do momento do pedido, cache de 60 s
- Retenção da chave em 24 h
- Nuvem, broker e gateway
- Salário de SRE, tributos e margem

Recomendação

30 Parar de errar: idempotência, snapshot e outbox, com o contrato atual intacto

60 Escalar e abrir: Catálogo em lote, broker, gateway, app no /v2

90 Expandir: segundo país (célula mínima só se a lei exigir) e parceiros em produção

Sem reescrita, sem janela de indisponibilidade, e a decisão mais arriscada já está provada em código.